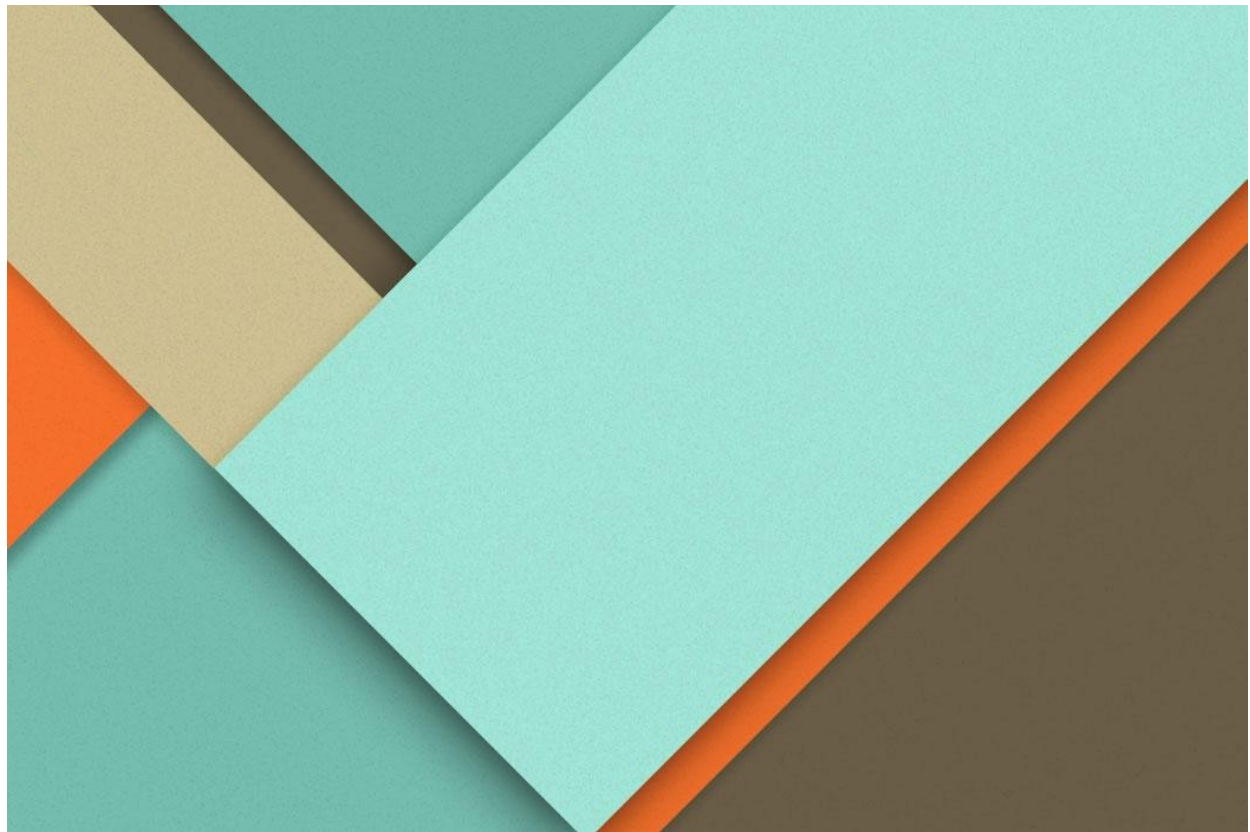




GOOGLE MAPS PROTOTYPE

06-01-2026

—

Ilyan khan

Rollno: 24CSE24

Subject: Data Structures and Algorithms.

Department: Computer Systems Engineering.

Quaid-Awam University of Engineering, Science and Technology, Nawabshah

Overview

For this project, I wanted to build a simple version of **Traffic Management System**. The main goal is to help users find the "quickest" path between two points (represented as P0, P1 etc.) based on the time it takes to travel through roads.

The Core Concept

The application treats the city as a **Directed Graph**. Each intersection or location is a "point" (node), and the roads connecting them are "edges." Since traffic or road conditions change, each road has a weight representing the travel time in minutes.

Data Structures Used

To keep things organized, I used the following:

- **Adjacency Matrix (cityMap[20][20]):** This 2D array stores the time taken to get from one point to another. If no road exists, it's set to a very high value (INT_MAX) to simulate infinity.
 - **Arrays for Pathfinding:**
 - parent[20]: Remembers the "previous" point in the journey so we can reconstruct the full route at the end.
 - dist[20]: Keeps track of the shortest time found from the start to every other point.
-

Functional Breakdown

The logic is contained within the TrafficApp class. Here is how each part of the code works:

1. The Constructor (TrafficApp)

When the app starts, it initializes the map. It sets every possible road to INT_MAX. This is important because, in graph algorithms, "infinity" represents that no path currently exists between two locations.

2. Adding Roads (addRoad)

This function allows the user to define a connection between two points. If a road already exists, calling this again simply updates the time, which is useful for simulating real-time traffic updates (like if a road gets congested and the time increases).

3. Finding the Route (findRoute)

This is the "brain" of the app. It uses the **Bellman-Ford Algorithm**.

- **Initialization:** It starts by setting the distance to the starting point as 0 and all others to infinity.
- **Relaxation:** It "relaxes" the edges by looping through all points multiple times. If it finds a shortcut (a path that takes less time than the one currently recorded), it updates the dist array and sets the parent for that point.

4. Displaying the Map and Path (showMap & showPath)

- **showMap:** Scans the matrix and prints out only the roads that have been added by the user. It ignores the INT_MAX values.
 - **showPath:** This is a **recursive function**. Since the parent array stores the path backward (e.g., "to get to P3, go through P2"), recursion helps us dive to the start and then print forward: P0 -> P1 -> P2 -> P3.
-

Main Logic and Conclusion

How the main() Function Works

The main() function handles the user interface. It uses a while loop combined with a switch statement to create a menu-driven experience.

- **Initialization:** It creates a TrafficApp object and pre-loads a few roads (like P0 to P1 taking 5 mins) so the map isn't empty when you first run it.
- **User Input:** The user can choose to view the live status, add their own roads, or calculate a route by entering start and end points.
- **Safety:** I added basic checks to ensure that if a user asks for a route that doesn't exist, the system tells them "No route found" instead of showing garbage values.

Personal Reflection

While developing this, I learned that the **Bellman-Ford algorithm** is great because it's relatively simple to implement with an adjacency matrix. Although it's slightly slower than Dijkstra's algorithm for very large maps, it's very reliable. It was cool to see how a few lines of logic can solve a complex problem like city navigation.

Future Improvements

If I were to expand this, I would:

1. Allow for a larger number of points dynamically using vectors instead of fixed arrays.
2. Add a feature to "block" a road entirely to simulate a road closure.
3. Implement a simple GUI to actually visualize the points on a map.

Few Weaknesses

1. Inefficient algorithm choice (not as fast as dijkstra algorithm)
2. Scalability issues due to use of arrays and not vectors.
3. A lot of wasted space due to fixed arrays.

Appendix: Full Source Code

```
#Open- Ended Lab Made by Ilyan Khan 24CSE24

#include <iostream>

#include <climits>

using namespace std;

class TrafficApp {
public:
    int points;
    int cityMap[20][20]; // mapp
    int parent[20];    // To know route

    TrafficApp(int n) {
        points = n;
        for (int i = 0; i < points; i++) {
            for (int j = 0; j < points; j++) {
                cityMap[i][j] = INT_MAX;
            }
        }
    }

    // Add or update road time
    void addRoad(int from, int to, int time) {
        if (from >= 0 && from < points && to >= 0 && to < points) {
            cityMap[from][to] = time;
        }
    }
};
```

```

    }
}

// Finding quickest path
void findRoute(int start, int end) {
    int dist[20];
    for (int i = 0; i < points; i++) {
        dist[i] = INT_MAX;
        parent[i] = -1;
    }
    dist[start] = 0;

    //bell man ford algorithm to find short path
    for (int k = 1; k < points; k++) {
        for (int i = 0; i < points; i++) {
            for (int j = 0; j < points; j++) {
                if (cityMap[i][j] != INT_MAX && dist[i] != INT_MAX) {
                    if (dist[i] + cityMap[i][j] < dist[j]) {
                        dist[j] = dist[i] + cityMap[i][j];
                        parent[j] = i;
                    }
                }
            }
        }
    }

    cout << "\n-----" << endl;
}

```

```
if (dist[end] == INT_MAX) {
    cout << "No route found from " << start << " to " << end << endl;
} else {
    cout << "Total Time: " << dist[end] << " mins" << endl;
    cout << "Route: ";
    showPath(end);
    cout << endl;
}
cout << "-----" << endl;
}

// Recursive function to print path
void showPath(int curr) {
    if (parent[curr] == -1) {
        cout << "P" << curr;
        return;
    }
    showPath(parent[curr]);
    cout << " -> P" << curr;
}

// Only show roads that exist
void showMap() {
    cout << "\n--- LIVE TRAFFIC STATUS ---" << endl;
    bool found = false;
    for (int i = 0; i < points; i++) {
        for (int j = 0; j < points; j++) {
```

```

        if (cityMap[i][j] > 0 && cityMap[i][j] < 1000000) {
            cout << "P" << i << " to P" << j << " | Time: " << cityMap[i][j] << "m" <<
endl;
            found = true;
        }
    }
}

if (!found) cout << "Map is empty!" << endl;
cout << "-----" << endl;
}
};

```

```

int main() {
    TrafficApp myApp(20);

    // Starting Roads
    myApp.addRoad(0, 1, 5);
    myApp.addRoad(0, 2, 12);
    myApp.addRoad(1, 2, 2);
    myApp.addRoad(2, 3, 1);
    myApp.addRoad(3, 4, 6);

    int choice;
    bool run = true;

    while (run) {
        cout << "----- TRAFFIC NAVIGATOR -----" << endl;

```

```
cout << "1. View Map" << endl;
cout << "2. Add/Update Road" << endl;
cout << "3. Find Best Route" << endl;
cout << "4. Exit" << endl;
cout << "Choose: ";
cin >> choice;
```

```
switch (choice) {
    case 1:
        myApp.showMap();
        break;

    case 2: {
        int u, v, t;
        cout << "From: "; cin >> u;
        cout << "To: "; cin >> v;
        cout << "Time: "; cin >> t;
        myApp.addRoad(u, v, t);
        break;
    }

    case 3: {
        int s, e;
        cout << "Start Point: "; cin >> s;
        cout << "End Point: "; cin >> e;
        myApp.findRoute(s, e);
        break;
    }
}
```



```
case 4:
    cout << "Closing system..." << endl;
    run = false;
    break;

default:
    cout << "Invalid choice! Try again." << endl;
    break;
}
}
return 0;
}
```